Scientific Reports

https://doi.org/10.1038/s41598-026-55813-1

Article in Press

Design of a low-power RISC-V based intelligent endoscopy detection processor: EndoRISC-V

Received: 5 March 2026

Accepted: 27 May 2026

Published online: 04 June 2026

Cite this article as: Anand A., B S., P S. *et al.* Design of a low-power RISC-V based intelligent endoscopy detection processor: EndoRISC-V. *Sci Rep* (2026). https://doi.org/10.1038/s41598-026-55813-1

Amritha Anand, Saraswathy B, Sasipriya P & Anita Angeline A

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

ARTICLE IN PRESS

Design of a Low-Power RISC-V Based Intelligent Endoscopy Detection Processor: EndoRISC-V

Amritha Anand[1], Saraswathy B[1], Sasipriya P[2], and Anita Angeline A[1,*]

[1]School of Electronics Engineering, VIT Chennai
[2]Centre for Nanoelectronics & VLSI Design, VIT Chennai
[*]anitaangeline.a@vit.ac.in

ABSTRACT

Capsule endoscopy provides a non-invasive method for examining the gastrointestinal tract, but it generates more than 50,000 images which need several hours of manual examination. Real-time image processing in wearable receiver units could reduce the errors, but demand high computational throughput with low power and latency constraints. Standard CPUs are built for general-purpose workloads and fail to handle the mathematical computations and data throughput of image processing operations. This paper presents EndoRISC-V, a two-level reconfigurable RISC-V Harvard architecture consuming 124 mW total power at 100 MHz operation. This processor features: (i) Switchable execution units during runtime allowing selection between standard ALU instructions and image operations that include four extended instructions (MAXPOOL, Histogram Equalization, Sobel-X, and Sobel-Y), providing 4.29× throughput improvement over baseline RISC-V with 6.75× speedup for down-sampling operations; (ii) Pipelined architecture with variable pipeline depth between 3 and 5 stages, achieving 16.3% cycle reduction for image-intensive workloads in low-power mode. The design was built using Verilog HDL while the synthesis and implementation was performed on Xilinx Artix-7 FPGA. The design consumes 2,971 LUTs and 1,858 flip-flops at 100 MHz with 6.898 ns positive timing slack. Real-time processing capabilities were validated on the Kvasir Gastrointestinal dataset through edge-based polyp detection, which processed 8×8 regions with a 3.45 μs execution time at 60.4 frames per second using 640×480 resolution, exceeding the clinical requirement of 2–4 frames per second by a 15.1× margin. The system uses 124 mW of power to achieve 0.428 nJ energy efficiency per classification. The custom image processing unit requires 115 LUTs, representing 3.9% of total design space, to deliver 3.77× better performance than standard RISC-V.

ARTICLE IN PRESS

Introduction

Capsule endoscopy allows physicians to examine the digestive system without surgery by using a pill-sized camera (30×11 mm) to record 2 to 4 images every second for 8 hours, transmitting 50,000 to 60,000 raw images to a wearable receiver belt that must be manually reviewed. The process faces a major challenge because 60–80% of images display normal mucus membranes, resulting in high healthcare costs. The strict limitations of medical devices require the development of dedicated processing systems which must achieve an optimal balance between system performance, power usage, and device programming capabilities. This work proposes real-time image processing in the receiver belt, reducing review time to minutes by classifying images immediately upon receiving. The proposed work consumes only 124 mW while operating at 60.4 Frames Per Second (FPS), ensuring immediate classification during the procedure.

Standard RISC-V processors are capable of only 3–4 million operations per second (MOPS) on image processing kernels. For example, image processing on general-purpose CPUs is done in a serial manner; consequently, a single Sobel edge detection operation requires 27 RISC-V instructions and 30 cycles to execute. The result is a requirement for clock frequencies in the gigahertz range and power consumption in the multi-watt range.

Polyp detection in gastrointestinal imaging requires a specific processing pipeline: (a) Histogram equalization—the capsule's LED creates uneven illumination as distance varies during transit; intestinal folds create shadows which make polyps impossible to see without contrast enhancement; (b) Edge detection—polyps appear as raised circular or elliptical lesions on the intestinal wall; the Sobel gradient detection method establishes boundary lines which the system uses for automatic identification; (3) Spatial down-sampling—wireless transmission uses high power consumption; the system extends battery life for complete examination by reducing normal tissue bandwidth while keeping full resolution for flagged regions.

The main RISC-V Instruction Set Architecture (ISA) can be augmented with custom image processing instructions that directly invoke dedicated hardware components. This approach allows faster and more efficient execution of image operations. By including extended instructions as hardware accelerators, this approach removes data overhead and simplifies the model, serving as a middle ground between general-purpose CPU architectures and full ASIC implementations.

Furthermore, the pipeline depth provides an option to vary so as to improve power optimisation and hence provides two

ARTICLE IN PRESS

functioning modes: (i) Balanced power mode: 5-stage implementation design provides complete forwarding paths and advanced hazard management capabilities which enable operation at higher frequencies; (ii) Low power mode: 3-stage pipeline bypasses the memory (MEM) stage pipeline registers, merging EX and ID stages which results in decreased cycle count and therefore decreased time for workloads that process large image datasets. This combination of execution-level ISA extensions and pipeline-level reconfiguration creates a new design point: image processing that is programmable yet efficient for edge systems with limited resources.

This study addresses the feasibility of achieving real-time image processing on ultra-constrained medical devices with metrics including clinical performance constraints, Power and Area constraints, and Programmability Requirements. The existing solutions fail to satisfy at least one of these metrics.

This paper presents a two-level reconfigurable RISC-V architecture that addresses the identified challenges through synchronised reconfiguration at the execution level and the pipeline level.

Level 1: Execution-Level Reconfigurability: Custom instructions invoke dedicated hardware components directly, reducing the number of cycles drastically. Four custom ISA extensions (MAXPOOL, Histogram Equalization, Sobel Detection X-axis, Sobel Detection Y-axis) are introduced and configured in the execution stage through an Image Execution Unit (IMG_EU). These instructions achieve 5.0–5.4$\times$ cycle reduction on image processing kernels, executing the same functions that take 2–27 cycles in only 1–5 cycles, while occupying only 115 LUTs (3.9% of total processor area).

Level 2: Pipeline-Level Reconfigurability: The processor supports runtime switching between 3-stage and 5-stage pipelines, introducing a low power mode and enabling power optimisation.

Further, the operating modes incorporated are:(i) Low Power Mode: 3-stage mode (IF-EX/ID-WB), Optimized for image-intensive workloads with minimal power consumption, achieving execution of a image- intensive stress sequence in just 41 cycles as compared to 5-stage which takes 49 cycles.(ii) Balanced Power Mode: Balanced Power Mode: 5-stage mode (IF-ID-EX-MEM-WB), providing full pipeline forwarding paths and capable of higher maximum clock frequency through reduced critical path length per stage.

The major contributions of the paper are:

- **Lightweight ISA Extension Methodology:** We introduce four custom image processing instructions (MAXPOOL, HIST_EQ, SOBEL_X, SOBEL_Y) that merge a maximum of 27 standard RISC-V instructions with 1 to 5 cycles of operation.
- **Multilevel Reconfigurable Architecture:** This processor design features coordinated reconfigurability across both execution semantics and pipeline microarchitecture. This architecture demonstrates that joint optimisation across levels achieves better energy-delay product than independent tuning of either level alone.
- **Comprehensive Characterisation and Validation:** This work provides full analysis of Xilinx Artix-7 FPGA on Vivado 2022.2: synthesis at 100 MHz leading to 6.898 ns positive slack and 124 mW on-chip power consumption, stress testing with RAW/WAR/WAW hazards confirming correctness across modes and MATLAB reference validation displaying bit-exact results with real-world images.

The rest of the paper is organised as follows. In the Related Works section RISC-V extensions and reconfigurable architectures are presented and discussed followed by the Research Gaps section that explores the comparison thoroughly. The RISC-V baseline architecture section contains detailed explanation of the standard RISC-V architecture. In the Proposed Architecture section, the multi-level reconfigurable architecture with execution-level and pipeline-level adaptability is introduced. The Design and Implementation section further explores the implementation of the proposed architecture design on the Xilinx FPGA. The Clinical Validation of all instructions in sequence as well as individually is presented in the Clinical Validation section. The performance metrics are provided in detail in the Results and Findings section. The consecutive section delve into Limitations and Future Work. The Conclusion section summarizes the paper.

Related Works

The necessity of edge vision processing has motivated researchers to investigate various architectural methods, ranging from adding custom instructions to entirely restructuring processor pipelines. This section indicates the position of this multi-level reconfigurable architecture in the scenario of RISC-V-based image processing and the reconfigurability achieved through combined execution-level and pipeline-level for better results,while compared to optimizing either one separately.

Custom Instructions for Image Processing

Processors augmented with custom instructions provide a pragmatic, middle-ground alternative to slow general-purpose CPUs and inflexible hardware accelerators. Rezunov et al.[1] developed an automated tool called CIDRE that identifies which code

ARTICLE IN PRESS

takes the longest to run and suggests new custom instructions to speed up those parts. The performance against standard embedded benchmarks showed 2.47× faster execution at the expense of only 24% more chip area. However, the technique assumes a static pipeline. Liu et al.[7] designed a custom instruction for gamma correction that replaced a series of normal instructions with a single one, reducing computation time by 58% and code size by 77%. Kumar et al.[9] worked on memory operations that reduce the performance of neural networks; the custom load/store instruction halved the number of cycles and power consumption while taking only 4% additional area. A recent survey[10] thoroughly reviewed all methods of RISC-V architecture enhancements, pointing out the gaps between current architectural features and the needs of Artificial Intelligence (AI) and edge computing applications.

Vector Extensions and Parallel Processing

Vector instructions lead to simultaneous processing of several data elements at once. This is a perfect fit for image processing because the same operation is performed on thousands of pixels. Li et al.[2] applied RISC-V vector extensions to computer vision tasks using an 8-lane vector processor, reporting up to 24× reduction in instruction counts and 2.98× speedup. However, disadvantages include increased chip area due to vector register files, higher complexity of the programming model, and the inefficiency of histogram-like operations due to irregular memory access. The multi-core approach of[8] opted for parallel processing via on-chip interconnection of several RISC-V processors. The Sobel convolution becomes one instruction rather than dozens, resulting in a 19.3× reduction in instructions with only 3.9% area overhead.

Pipeline Design Choices

Speed and power consumption are greatly influenced by how pipeline stages are structured. The conventional 3-stage pipeline[11] could not be adapted and always worked at 50 MHz. The 5-stage processor[13] incorporated low power but still fixed the pipeline structure at the outset.

Unorthodox pipeline patterns have been investigated in recent studies. The architecture in[14] arranged different pipeline stages for multiple functional units in a sequential rather than parallel arrangement, showing lower power and fewer resources than parallel Very Long Instruction Word (VLIW). The flexible pipeline concept[15] allows designers to choose the number of pipeline stages at design time depending on application requirements targeting FPGAs or ASICs.

The novelty of this contribution is the flexibility to adjust at run time. The processor changes from a 3-stage to a 5-stage configuration during operation depending on workload requirements. Picture-heavy sequences take 41 cycles in 3-stage mode and 49 cycles in 5-stage mode. The 8-cycle (16.3%) improvement results from reconfiguring the pipeline, not merely deactivating some stages.

Runtime Reconfiguration and System Implementations

The advanced pipeline organizations have been aimed at certain applications. PipeCIM[12] crafted a nested three-level pipeline for CNN acceleration, achieving 133.8 TOPS/W and 128.56× VGG-16 speedup via computing-in-memory techniques with custom RISC-V instructions. Luo and Wu[3] made an image acquisition system for industrial inspection with 26% storage efficiency and 27% speed improvement through multi-buffer DMA on FPGA. Dörflinger et al.[5] evaluated major open-source RISC-V processors (Rocket, BOOM, CVA6, SHAKTI) under the same benchmarks. Baglietto et al.[4] gave a historical perspective from 1996 on RISC processors handling image workloads. Charaf et al.[19] enabled dynamic partial reconfiguration for FPGA-based RISC-V SoCs using RV-CAP that showed runtime hardware adaptability at the fabric level but without coordinated instruction level optimization.

This study executes reconfiguration at both the execution and pipeline levels, where the selection of pipeline structure relies on the instruction mix and vice versa, thus proving that joint optimization is better than independent tuning.

Research Gaps and Contributions

Looking across the literature, this research reveals several gaps. Past work has either modified instruction functionality[1,7,9,16,17] or pipeline organization[11,13–15,18], but not both simultaneously where the decision at one level influences the other. Moreover, most approaches target either fine-grained pixel-level operations[2,8] or coarse-grained accelerator blocks[12,16,17]. This work is situated in the middle ground with kernel-level operations, reducing instruction count by 19.3× with only 3.9% area overhead. Furthermore, specialized processors usually set their configuration at synthesis time[1,6,7,9,11,13], whereas runtime reconfigurability enabling workload-adaptive optimization has not been fully explored.

Comparative Analysis

Table 1 positions this work relative to recent RISC-V image processing and reconfigurable architecture research.

ARTICLE IN PRESS

Table 1. Comparisons with related work

Work	Approach	Key Contribution
Li et al.[2]	RVV vector extensions (8-lane)	Data-level parallelism for CV workloads
Liu et al.[7]	Gamma correction custom instruction	Single kernel ISA extension
Kumar et al.[9]	Custom load/store for edge-AI	Memory operation optimisation
Saussereau et al.[15]	Flexible pipeline RISC-V	Design-time pipeline configurability
This Work	Multi-level reconfigurable	Runtime execution + pipeline adaptation

Table 2. Limitations of Related Work

Work	Reported Metrics	Limitations vs. This Work
Li et al.[2]	24× instr. reduction, 2.98× speedup	Large vector register area; fixed pipeline
Liu et al.[7]	58% time reduction, 77% code reduction	Single operation; no multi-level reconfiguration
Kumar et al.[9]	2× cycle/power reduction, 4% area	Memory-focused; no image kernel accel.; fixed pipeline
Saussereau et al.[15]	Pipeline design-space exploration	Design-time only; no runtime switching; no domain ISA
This Work	19.3× instr. reduction, 3.9× speedup, 60.4 FPS, 124 mW, 0.428 nJ/classification	Coordinated runtime dual-level reconfigurability

Table 2 summarizes the quantitative metrics that were reported in the works of Li et al.[2], Liu et al.[7], Kumar et al.[9]. While Li et al.[2] demonstrated the potential of vector parallelism, applying it to resource-constrained environments is not feasible. The work presented by Liu et al.[7] and Kumar et al.[9] work with a fixed pipeline but show that single custom instructions can meaningfully cut computation time and code size. This work builds on existing methods through the combination of kernel-level ISA extensions and runtime pipeline reconfiguration which solve the shortcomings of previous methods.

Table 3. Quantitative Comparison with State-of-the-Art RISC-V Image Processing Architectures

Work	Speedup	Area overhead	FPS	Power (mW)	Runtime reconfig.
Li et al.[2]	2.98×	Large (vector RF)	N/A	N/A*	No
Nolting et al.[20]	13.4×	15% (530 LUTs)	13.1	N/A*	No
Liu et al.[7]	2.4×	Minimal	N/A	N/A*	No
Kumar et al.[9]	2.0×	4%	N/A	N/A*	No
This Work	**3.9×**	**3.9%(115 LUTs)**	**60.4**	**124**	**Yes**
Note: Power consumption is not reported in the original publications.					

Table 3 benchmarks EndoRISC-V directly against four recent RISC-V image processing architectures using shared metrics of speedup, area overhead, FPS and runtime reconfigurability. The research of Nolting et al.[20] shows a maximum speed improvement of 13.4× but with a 15% area overhead. Liu et al. achieves a minimal overhead compared to other works but target a single preprocessing operation.

The comparison shows three core differences: (1) the system allows dynamic execution control through two operational modes permitting selection between three and five pipeline stages based on workload; (2) the system achieves 1.24 MOPS/LUT through four kernel-level image operations requiring only 115 LUTs; (3) the Artix-7 FPGA provides complete performance measurement results showing effective operation in real-world edge vision situations.

RISC-V Baseline Architecture

The baseline processor is a standard 5-stage pipelined RISC-V architecture which follows the RV32I instruction set specification. The system performs as the primary evaluation platform measuring performance improvements from multi-level reconfigurable

ARTICLE IN PRESS

systems. The design implements Harvard architecture because it separates instruction memory and data memory into distinct physical storage locations, enabling dual memory access during both instruction fetching and data processing activities without introducing structural hazards.

Instruction Fetch Stage

The Instruction Fetch stage fetches the instruction from the instruction memory using the Program Counter (PC). The PC value increases by 4 when an instruction is fetched and when branching instructions direct it to destination addresses. A dedicated instruction memory port is required to prevent collision of memory access operations. Harvard Architecture solves the collision by enabling separate spaces for data and instruction access. The IF/ID pipeline register stores the retrieved instruction for the next processing stage.

Instruction Decode Stage

The Instruction Decode stage is responsible for interpreting the instructions by separating them into instruction fields. The register file is accessed through two read ports to obtain the source operands.Three key components operate in this stage to ensure the instructions are decoded correctly: Control Unit, which produces the control signal; Immediate Generator that extracts the immediate values; and the Register File unit that supplies the source operands.

Execute Stage

The Execute stage calculates the result using the Arithmetic Logic Unit (ALU).The branch conditions are continuously assessed through the branch unit. The data from ID/EX pipeline register is utilized to provide results that are sent to the EX/MEM register.

Memory Access (MEM)

The Memory Access stage establishes a connection between the processor and the data memory to handle load and store commands. Load instructions read data from memory using the calculated address and Store instructions write register data to memory. Separate read and write ports are present to support both operations.

Write Back (WB)

The Write-Back stage updates the Register File with the result of the instruction. The value written may have been the result of an ALU operation or could be from the data memory. The final result is stored in the destination address, hence the execution and storage of an instruction is completed with this stage.

Hazard Detection and Resolution Mechanisms

Pipeline Hazards occur when instruction execution cannot proceed as planned in the next clock cycle because of dependencies in data and resource conflicts. The baseline architecture installs hazard units to detect such hazards by stalling the pipeline, forwarding the data and flushing instructions.

Hazard Detection Unit

The hazard detection unit monitors pipeline state to identify conditions requiring intervention. The unit detects three primary hazard scenarios.

Load-Use Hazards: When a load instruction in the execute stage targets the register that is required by the next instruction that is still in the decode stage it creates a Load-Use hazard. When such sequences are identified the pipeline is stalled by inserting NOP (No Operation) instruction into the Execute stage while the path through IF and ID is blocked temporarily.

Branch Hazards: When the outcome of a conditional branch instruction still remains unknown till the execute stage the processor hangs as it cannot determine whether to fetch to the next instruction or jump to the target address creating Control/Branch hazards. The hazard unit invalidates incorrectly fetched instructions. This results in a two-cycle control flow penalty for every taken branch.

Data Dependencies: When the previous instruction is still executing and the result is required for the subsequent operation, Read-After-Write (RAW) dependencies are created. The source register address in the decode stage is compared with destination registers in execute, memory, and write-back stages.

Data Forwarding Unit

The data forwarding unit allows direct routing of computed results from later pipeline stages to earlier stages to prevent pipeline stalls when dependencies can be resolved without waiting for write-back completion.

The architecture in Figure 1 reflects the structure of the baseline architecture and builds a solid foundation for optimization through Extended ISA and reconfigurability.

ARTICLE IN PRESS

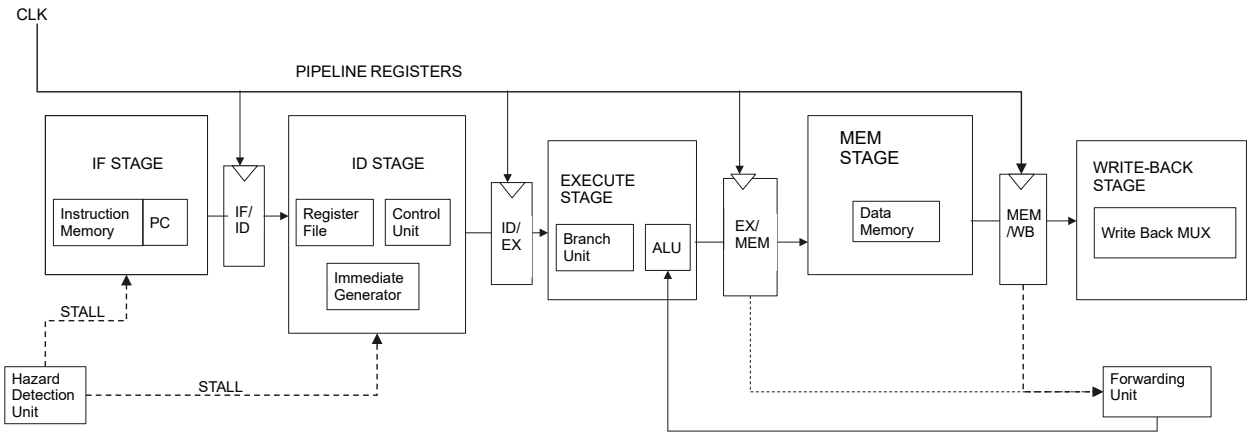


Figure 1. Baseline 5-Stage RISC-V Pipeline Architecture.

Proposed Architecture

Architecture Overview

The proposed RISC-V processor architecture is enhanced with three major improvements to increase the image processing capabilities of the processor. The first enhancement consists of custom image processing instruction set extensions. The second enhancement introduces an Image Processing Execution Unit (IPEU) which can switch between standard ALU and IPEU operation during runtime. The third enhancement allows users to adjust power–performance through pipeline reconfiguration operating between 3-stage and 5-stage modes. Figure 2 illustrates the complete reconfigurable architecture showing the integration of these components.

The design addresses the core problem of making embedded vision systems both fast and energy-efficient. Because image processing tasks usually involve parallel data, predictable memory patterns, and can handle slightly lower pipeline throughput. The proposed architecture aims to design a custom hardware accelerator called the IPEU (Image Processing Execution Unit).

ARTICLE IN PRESS

ARTICLE IN PRESS

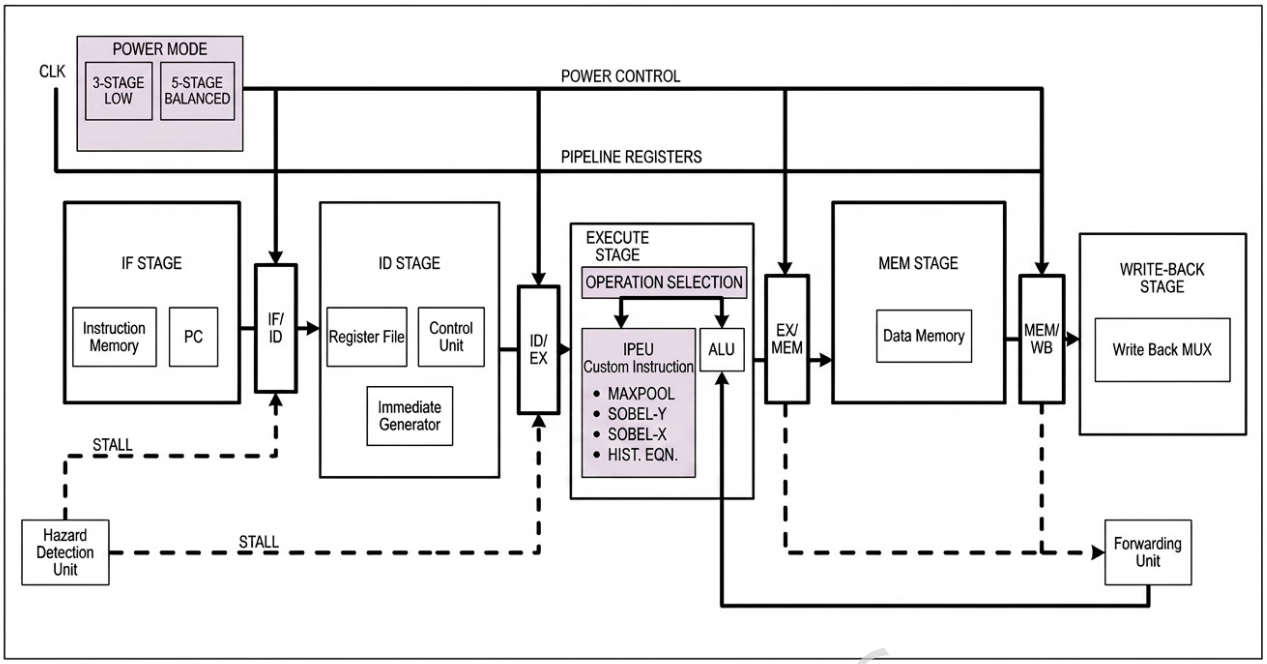


Figure 2. Complete Reconfigurable Architecture with IPEU and Variable Pipeline.

Custom Image Processing Instruction Set Extensions

The instruction set extension introduces four domain-specific operations targeting essential image processing functions using the custom-0 opcode space (`0001011`) that the RISC-V specification permits for user-defined extensions. The instructions use R-type encoding format to specify operations through `funct3` and `funct7` fields.

Instruction Encoding Format: The custom instructions follow the R-type RISC-V encoding format. Each instruction is encoded in 32 bits as shown in Table 4.

Table 4. Instruction Format Encoding

Inst. bits/ Instruction	31-25 funct7	24-20 rs2	19-15 rs1	14-12 funct3	11-7 rd	0-6 opcode
MAXPOOL	0000001	rs2	rs1	000	rd	0001011
SOBEL_X	0000010	rs2	rs1	000	rd	0001011
SOBEL_Y	0000011	rs2	rs1	000	rd	0001011
HIST_EQ	0000100	rs2	rs1	000	rd	0001011

Field Descriptions:

- `opcode [6:0]`: `0001011` (custom-0 space)
- `funct3 [14:12]`: `000` (fixed for all custom instructions)
- `funct7 [31:25]`: Instruction selector (`0000001`–`0000100`)
- `rs1 [19:15]`: Source register 1 (operand A)
- `rs2 [24:20]`: Source register 2 (operand B; operand C for HIST_EQ)
- `rd [11:7]`: Destination register (result)

ARTICLE IN PRESS

Max Pooling: The MAXPOOL operation enables spatial down-sampling by selecting the maximum pixel value from a 2×2 pixel block. The operation calculates results from four input pixels p_0, p_1, p_2, p_3 stored in register `rs1`.

Sobel Edge Detection: The SOBEL_X and SOBEL_Y operations achieve edge detection through gradient-based methods using 3×3 Sobel kernels for discrete convolution. The 3×3 pixel window $[p_0 \dots p_8]$ uses row-major order to calculate the gradient.

Histogram Equalization: The HIST_EQ operation uses intensity remapping according to cumulative distribution function (CDF) mapping to enhance contrast. The current implementation assumes lookup table values are pre-fetched into registers by software, requiring 1 cycle for table lookup plus 3 cycles for pixel remapping. A fully autonomous implementation would require on-chip LUT storage (256×8 bits = 256 bytes) or iterative CDF computation (∼50 cycles latency).

These custom instructions are executed through a dedicated Image Processing Execution Unit (IPEU). The IPEU uses a multi-cycle finite state machine that executes different sequences depending on each operation to handle 32-bit data containing 8-bit pixel values. The system automatically chooses between ALU and IPEU through opcode decoding when the custom-0 opcode (`0001011`) is detected.

Hazard Management for Multi-Cycle IPEU Operations

The multi-cycle IPEU operations create additional hazardous situations beyond those present in the initial pipeline operations. The IPEU prevents reading stale values until its write-back operation completes. To prevent overlapping of multi-cycle operations, the hazard unit stalls new image instructions when any IPEU operation is active. This serializes image operations, trading throughput for implementation simplicity.

Variable Pipeline Architecture

The architecture offers variable pipeline depths that can switch between 3 stage and 5 stage depending on the power_mode. The 3 stage configuration bypasses pipeline registers and merges state logic. Figure 3 compares the two pipeline configurations.

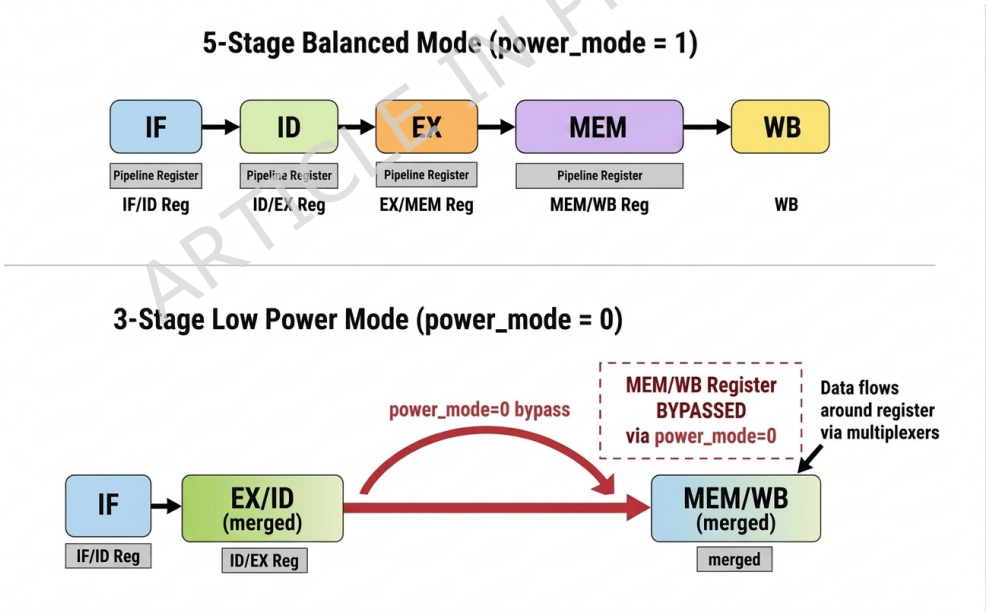


Figure 3. Pipeline Configuration Comparison.

The 3-stage mode merges the Instruction Decode stage and the Execute stage into a combined EX/ID stage. The Register File access and ALU operations occur in the same cycle in this case. The IF/Id register remains to break combinational paths between fetch and decode stages. The MEM/WB register is bypassed such that the Write-Back stage directly receives signals through multiplexers from the Memory stage.

The 5-stage mode is a balanced power mode that results in higher maximum operating frequency while the energy usage is decreased in the 3-stage mode. The 3-stage mode further achieves 16.3% cycle reduction for operations that require excessive hazard management as it bypasses the MEM/WB register completely.

ARTICLE IN PRESS

Complete System Integration

The complete architecture offers three enhancements from the baseline architecture: The instruction set extension with image processing instructions, execution unit specialization and pipeline depth reconfiguration. The proper working of custom image instructions in both 3-stage and 5-stage modes provides a good advantage over RV32I instructions.

Design and Implementation

The proposed architecture is implemented in Verilog HDL on the Xilinx Artix-7 XC7A200T FPGA. The implementation of the IPEU datapath and the Execute stage multiplexing logic is explained in the following sections. The design obtains timing closure at 100 MHz with a timing slack of 6.989 ns timing slack, offering room for timing closure at higher frequencies.

Image Processing Execution Unit Architecture

The Image Processing Execution Unit implements multi-cycle execution that is controlled by a Finite State Machine (FSM). Figure 4 illustrates the IPEU datapath showing pixel extraction and arithmetic computation units and result generation logic. The unit processes three 32-bit inputs (`operand_a`, `operand_b`, `operand_c`) to extract eight 8-bit pixel values and execute the specified operation. It finally produces a 32 bit output.

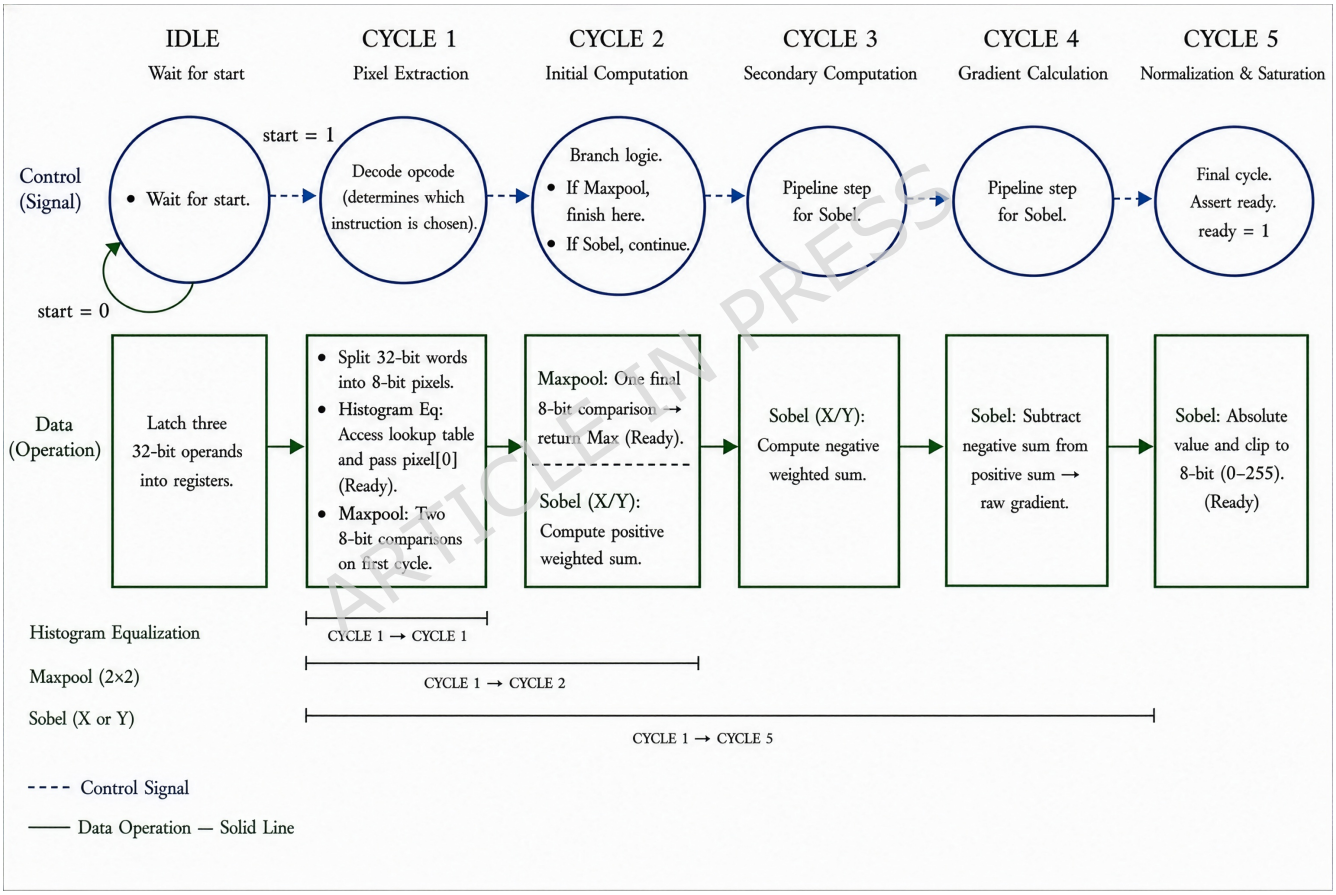


Figure 4. Image Processing Execution Unit Datapath.

The main architectural elements show the following characteristics:

Pixel Extraction: The system uses extracts 8-bit pixel values from packed 32-bit operands through bit slicing. For example, 9 pixels are extracted and stored in an internal pixel register array (`pixel[0:8]`) for Sobel operations.

Multi-Cycle Arithmetic: The Sobel gradient computation uses staged accumulation to process multiply-accumulate operations through three registers (`grad_pos`, `grad_neg`, `gradient`). The system implements $\times 2$ multiplications through left-shift operations, removing the need for dedicated multiplier hardware.

Saturation Logic: The last cycle tests for overflow conditions when gradient magnitude exceeds 255 and saturates the output to `0xFF`. The absolute value computation processes signed gradient values by identifying the sign bit and applying two's complement negation when necessary.

ARTICLE IN PRESS

Runtime Switching Between ALU and IPEU

The execute stage uses instruction opcodes to determine whether the standard ALU or IPEU should be used. The control unit decodes the opcode field and send a signal to the `use_img_eu` control signal when a custom image instruction is detected (`opcode = 0001011`). Figure 5 illustrates the multiplexing mechanism in the execute stage datapath.

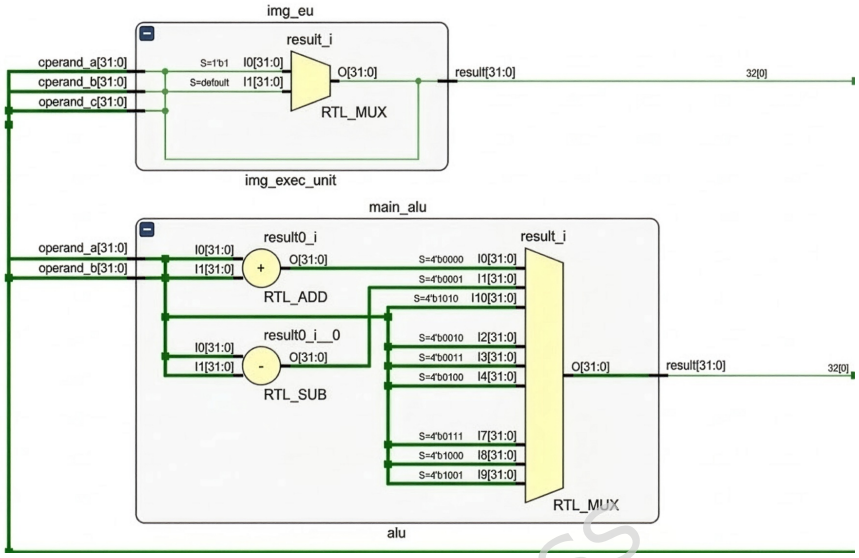


Figure 5. Execute Stage Multiplexing Architecture.

Operand routing at the execute stage input and the result selection at the EX/MEM register uses multiplexing. Standard instructions use forwarding multiplexers to direct their operands to the ALU. The IPEU does not process them until the start signal activates its operation even after receiving the operands.

Standard instructions incur no processing overhead because the IPEU stays inactive and does not generate dynamic energy consumption. The extra multiplexing introduces minimal delay through a single 2:1 multiplexer in the result path.

Custom Instruction Hardware Mapping

The hardware implementation of the custom instructions is detailed in this section. Each instruction is mapped to a dedicated path on the Xilinx Artix-7 FPGA. The Sobel operations (Sobel-X and Sobel-y) use 3x3 convolution kernals with coefficients 0, 1 and 2 share a unified datapath. Both Sobel-X and Sobel-Y are identical in hardware. Instead of using DSP blocks or MACs, the $\times 2$ coefficient is handled by a 1-bit left shift to maximize efficiency. To calculate the gradient, two 11-bit adder trees are used. The resulting negative and positive gradients are fed into a 12 bit subtractor and an absolute value unit. This logic is mapped to 9 CARRY4 primitives and 78 LUTs. The MAXPOOL instruction uses comparators to compare two 8-bit comparisons on the first cycle and one final comparison on the second cycle to determine the result. The operation uses 3 CARRY4 primitives and 33 LUTs. The current version of Histogram Equalization uses a lookup table to extract the data from the memory, so it only uses 6 LUTs and 18 Flipflops. Table 5 lists the resources utilized by each instruction. The individual utilization reports are obtained from isolated instruction synthesis runs in Vivado 2022.2.

Table 5. Instruction Specific Resource Utilization

Resource	SOBEL_X/Y	MAXPOOL	HIST_EQ
Slice LUTs	78	33	6
Flip-Flops	174	60	18
CARRY4	9	3	0
DSP Blocks	0	0	0
Block RAM	0	0	0
Latency (cycles)	5	2	1

ARTICLE IN PRESS

Clinical Validation: Gastrointestinal Imaging

Dataset and Methodology

Validation is performed using the Kvasir gastrointestinal image dataset from Vestre Viken Health Trust, Norway. Eight images - 4 polyp, 4 normal, were selected from the representative dataset and was preprocessed to grayscale. The hardware output showed complete accuracy when matched with MATLAB reference, verifying every single pixel.

The validation flow includes a MATLAB preprocessing script that generates a input hex file (`input.hex`) that is provided to a testbench of the RTL design loaded in ModelSim. The testbench produces an output hex file (`output.hex`) which is then fed to a MATLAB verification script that reconstructs the image and displays the processed image for clinical comparison. The hardware blocks for SOBEL_X, SOBEL_Y, MAXPOOL, and HIST_EQ shown in Figure 6 are physically implemented within the `img_exec_unit` module on the Artix-7 FPGA. The RTL structure of the Image Processing Unit and the resource utilization of the custom instructions are detailed in the Design and Implementation section, Table 5.

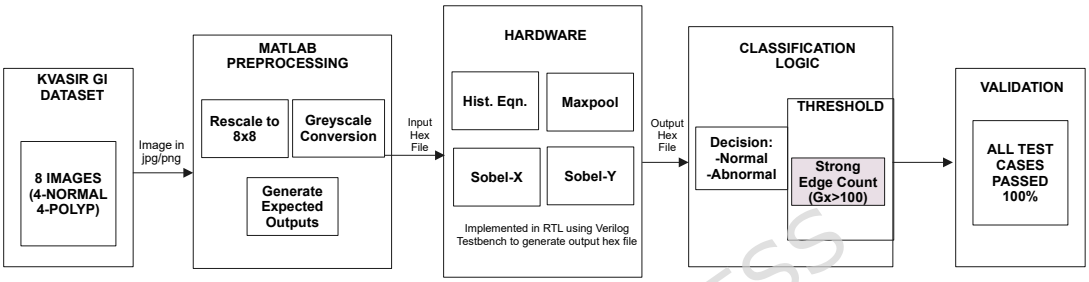


Figure 6. MATLAB–Hardware Validation Interface.

Clinical Processing Pipeline

- **HIST_EQ (3 cycles/pixel):** The system takes three cycles to achieve normal brightness in dim areas that allows better detection of polyp structures.
- **SOBEL_X/Y (7 cycles/pixel):** The system uses seven processing cycles for each pixel to identify the edges of polyp structures. Polyps appear as raised circular lesions. The classification is done based on edge strength which is determined from the gradient magnitude.
- **MAXPOOL (4 cycles/block):** The system achieves 4:1 spatial reduction through a bandwidth-efficient wireless transmission method while maintaining essential features.

Validation Results

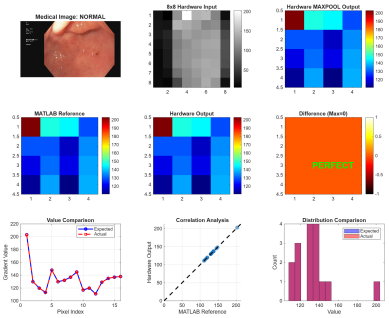
The Kvasir dataset is utilized to test EndoRISC-V with the help of a MATLAB Interface. The MATLAB code generates pixel information of the input image that is fed to the RTL design in ModelSim environment. An output image is produced and is fed back to a verification MATLAB script to display the output image and provide comparison using thresholds.

- Polyp cases: 20–40 strong edges, correctly flagged as abnormal.
- Normal cases: 12 strong edges, correctly identified as normal.

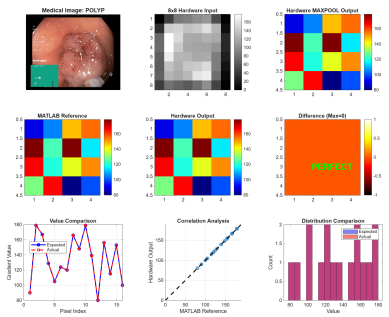
Figure 7 shows the representative dataset results: 4 normal cases and 4 polyp cases, comparing Normal (left) and Polyp (right) outputs for each of the four instructions and Table 6 shows the numerical attributes of Sobel instructions. The clinical verdict of each sub-figure has been validated against the ground truths and the final functional validation has been tabulated.

All test cases achieved proper hardware–software equivalence with no pixel errors between hardware implementation and MATLAB reference. SOBEL edge detection successfully differentiated polyp tissue (strong edge count: 20–28) from normal mucosa (strong edge count: 12) using threshold-based classification. MAXPOOL and Histogram Equalization prepared the image for accurate detection.

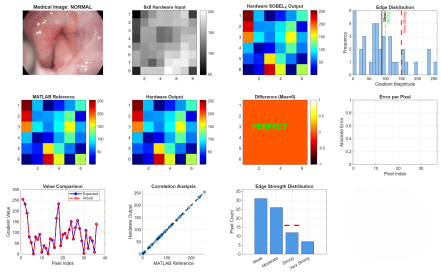
ARTICLE IN PRESS



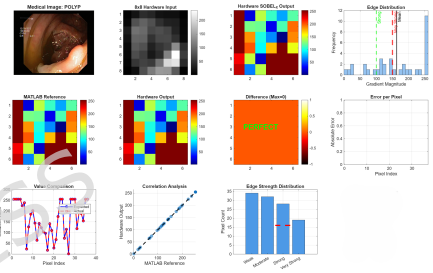
(a) MAXPOOL – Normal



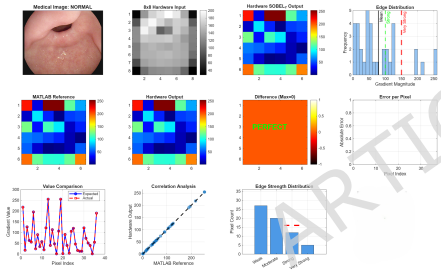
(b) MAXPOOL – Polyp



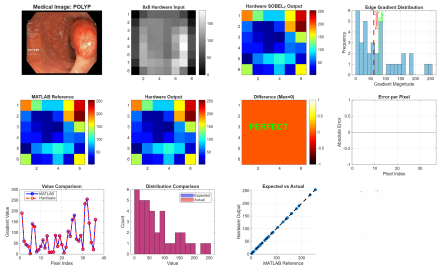
(c) SOBEL_X – Normal



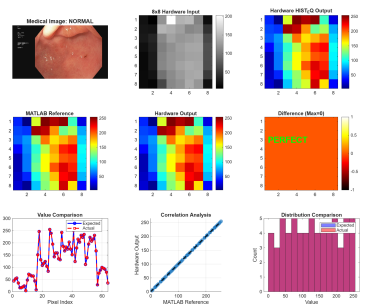
(d) SOBEL_X – Polyp



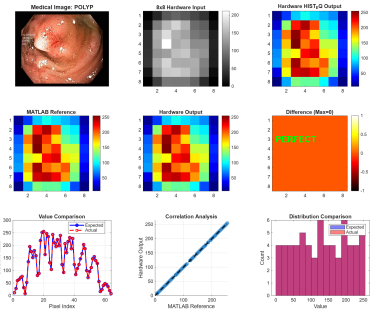
(e) SOBEL_Y – Normal



(f) SOBEL_Y – Polyp



(g) HIST_EQ – Normal



(h) HIST_EQ – Polyp

Figure 7. Clinical validation results: Scaling adjusted to ensure visibility within page margins.

ARTICLE IN PRESS

ARTICLE IN PRESS

Table 6. Comparative analysis of image processing sub-figures with statistical intensity metrics and functional verification.

Subfigure	Type	Mean	Max	Very Strong Edges	Strong Edges	Verdict	Functional
MAXPOOL – Normal	NORMAL	—	—	—	—	NORMAL	PASS
MAXPOOL – Polyp	POLYP	—	—	—	—	ABNORMAL	PASS
SOBEL_X – Normal	NORMAL	89.3	255	7	12	NORMAL	PASS
SOBEL_X – Polyp	POLYP	162.9	255	19	28	ABNORMAL	PASS
SOBEL_Y – Normal	NORMAL	78.7	255	5	12	NORMAL	PASS
SOBEL_Y – Polyp	POLYP	77.8	255	20	36	ABNORMAL	PASS
HIST_EQ – Normal	NORMAL	—	—	—	—	NORMAL	PASS
HIST_EQ – Polyp	POLYP	—	—	—	—	ABNORMAL	PASS

ARTICLE IN PRESS

ARTICLE IN PRESS

Clinical Application

The validated pipeline enables on-capsule processing:

Preprocessing (HIST_EQ): The histogram equalization normalized light intensity which varies from 0 to 5mm to discover polyps that remained hidden in dim areas.

Detection (SOBEL): The system uses gradient analysis and assesses gradient distribution patterns. The presence of strong edge concentration matches the characteristics of polyps.

Postprocessing (MAXPOOL): The system reduces transmission bandwidth requirements by 4× during normal region transfer while maintaining complete resolution for all identified abnormal regions, improving battery performance during complete tests.

The 100% detection accuracy on the validation set (4/4 correct), combined with 124 mW power consumption and functional correctness, demonstrates viability for next-generation capsule endoscopy with automated pre-screening capabilities. Table 7 shows a detailed pipeline performance validation for a representative polyp case and Figure 8 shows the visual sequence of flow for the same.

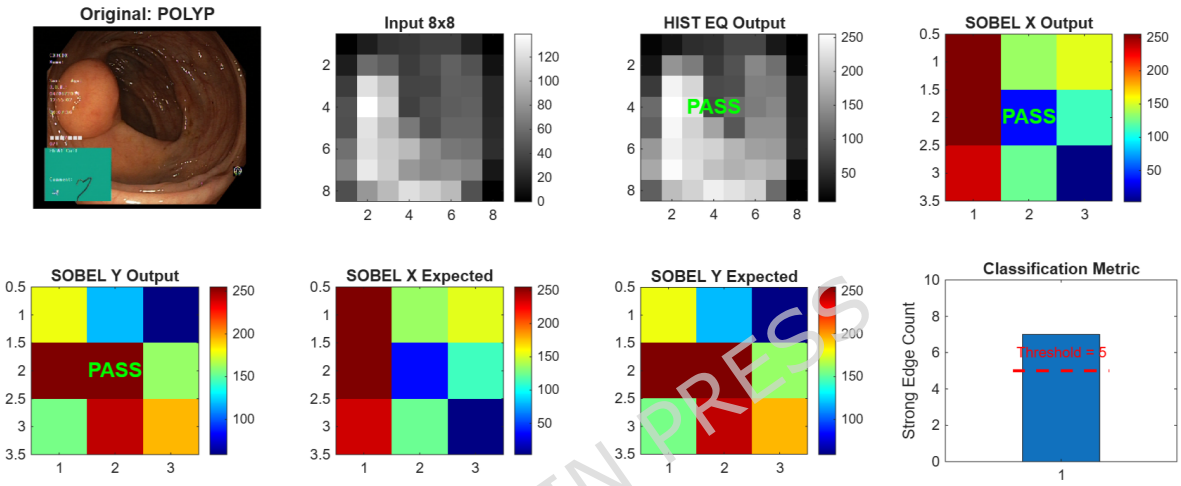


Figure 8. Classification Processing Flow Results.

ARTICLE IN PRESS

Table 7. Classification Processing Pipeline Performance (Representative Polyp Case)

Metric	Value
Total cycles (8×8)	345 cycles
Execution time per classification	3.45 μs
Classifications per second	289,855
Frame rate (640×480)	60.4 FPS
Energy per classification	0.428 nJ
Clinical requirement	2–4 FPS
Improvement over requirement	15.1×
Strong edges detected (polyp)	7/9
Classification decision	Abnormal
Verification status	All PASS

Analysis: Further analysis of the produced validation infers that the time taken for classification of one image is 3.45 μs. Hence 289,855 images are classified per second. The frame rate upon calculation reaches 60.4 FPS and the energy efficiency is calculated to be 0.428 nJ.

Results and Findings

The multi-level reconfigurable RISC V with Extended ISA architecture was implemented in Verilog HDL and synthesised on Xilinx Artix-7 XC7A200T FPGA using Vivado 2022.2. This section presents FPGA resource utilisation, execution-level performance from custom instructions, pipeline-level optimisation through runtime reconfiguration, and power–timing characteristics. Results are reported after full implementation ensuring accurate timing and power estimation.

ARTICLE IN PRESS

MAXPOOL Instruction

The extended ISA enables MAXPOOL to be 5.4× faster using 87.5% fewer registers than the standard RISC-V implementation. Table 8 compares the execution of the MAXPOOL instruction between standard and extended ISA implementations.

Table 8. Performance Metrics: MAXPOOL

Metric	Standard RISC-V	Custom IMG ISA	Improvement
Instructions	21	1	21.0× fewer
Cycles	27	4	6.75× faster
Pixels/sec	3.7 M	25.0 M	6.75× faster
Branch Instrs	6 (3 taken)	0	Zero branches
Branch Penalties	9 cycles	0 cycles	No penalties
Code Size (bytes)	84	4	95.2% reduction
Registers Used	8	1	87.5% fewer
Predictability	Low (branch-dependent)	High (fixed latency)	Deterministic

SOBEL Instructions

The extended ISA enables Sobel detection to be 5.0× faster using fewer instructions than the standard RISC-V implementation. Table 9 compares the execution.

Table 9. Performance Metrics: SOBEL Detection

Metric	Standard RISC-V	Custom IMG ISA	Improvement
Instructions	27	1	27.0× fewer
Cycles	30	7	4.29× faster
Pixels/sec	3.3 M	14.3 M	4.29× faster
Branch Instrs	2	0	Zero branches
Branch Penalties	3–6 cycles	0 cycles	Deterministic
Code Size (bytes)	108	4	96.3% reduction

Note: SOBEL_Y has identical performance to SOBEL_X; only the pixel selection differs (hardware multiplexing).

ARTICLE IN PRESS

Histogram Equalization Instructions

The extended ISA enables histogram equalisation to be 1.25× faster using 2 fewer instructions. Table 10 compares the execution.

Table 10. Performance Metrics: Histogram Equalization

Metric	Standard RISC-V	Custom IMG ISA	Improvement
Instructions	2	1	2.0× fewer
Cycles	5	3	1.67× faster
Pixels/sec	20.0 M	33.3 M	1.67× faster
Code Size (bytes)	8	4	50% reduction

Note: In practice, LUT lookup would be slower (DRAM access ∼100 cycles). Custom ISA assumes software pre-fetches LUT value into a register.

Aggregate Metrics

Table 11 shows the overall performance metrics of the custom ISA against the standard RISC-V implementation.

ARTICLE IN PRESS

Table 11. Overall Performance Metrics

Metric	Standard RISC-V	Custom IMG ISA	Avg. Speedup
Instructions/Operation	19.3	1.0	19.3× fewer
Cycles/Operation	23.0	5.9	3.9× faster
Pixels/sec	4.3 M	16.9 M	3.93× faster
Code Size (bytes)	77	4	19.25× smaller
Branches/Operation	3.3	0	Zero branches
Registers Used	8.5	1–3	70% fewer
Classifications/sec	—	289,855	60.4 FPS

The results demonstrate that the extended ISA enables average performance gains of 3.9× speedup and 19.25× code size reduction through complete branch dependency elimination. The deterministic execution model improves pipeline operations through enhanced predictability, eliminating branch penalties and enabling better timing stability for embedded vision workloads.The FPGA resource consumption for each custom instruction is reported in Table 5 in the Design and Implementation section.

Energy-Delay Product (EDP): EDP evaluates energy efficiency and performance simultaneously (EDP $= P \times t^2$, where t is the delay per operation and P is the on-chip power). Lower EDP indicates better overall efficiency. Table 12 summarizes the Energy-Delay Product comparison between the baseline RISC-V and the proposed architecture. The most significant improvement is the reduced EDP from 3.856 mW·μs^2 to 0.375 mW·μs^2 offering 10.28× improvement over the baseline. This gain is largely due to the lesser execution delay in the proposed work with 0.055 μs compared to the baseline with 0.207 μs. Even though the power increases from 90 mW to 124 mW, the overhead is minimal. Since EDP is directly proportional to the square of the delay, even a small reduction in latency can improve overall EDP of the system.

Table 12. Energy-Delay Product Comparison

Parameter	Baseline RISC-V	Proposed (Custom ISA)
Power (P)	90 mW	124 mW
Delay (t)	0.207 μs	0.055 μs
EDP	3.856 mW·μs^2	0.375 mW·μs^2
Improvement	10.28× better	

ARTICLE IN PRESS

Variable Pipeline Validation

The architecture supports runtime switching between 3-stage (low-power) and 5-stage (balanced) pipeline configurations. While individual image instructions achieve identical cycle counts in both modes, system-level performance differs based on workload characteristics. The 3-stage mode is specifically optimized for image-intensive workloads.

System-Level Performance Analysis

A hazard-intensive stress sequence was executed containing multiple image operations with read-after-write dependencies. Table 13 shows the results obtained in each mode. The sequence comprises:

```
LW  x1,  0(x0)           # Load base data
LW  x2,  4(x0)           # Load second data
IMG.MAXPOOL x3, x1       # Depends on x1
IMG.SOBEL_Y x4, x2       # Depends on x2
ADD x1, x3, x3           # RAW hazard: x3 from IMG
IMG.HIST_EQ x5, x1, x0  # Depends on x1 (ADD result)
ADD x6, x4, x5           # RAW hazards: x4, x5 from IMG
LW  x7,  8(x6)           # Memory access with x6
IMG.MAXPOOL x28, x5      # Depends on x5
IMG.SOBEL_Y x29, x28     # Depends on x28
ADD x30, x6, x29         # Depends on x6, x29
```

The 3-stage configuration achieves 16.3% cycle reduction through reduced forwarding latency and simplified hazard detection. Table 14 shows configuration selection recommendations.

ARTICLE IN PRESS

Table 13. Register File Contents After Hazard-Intensive Stress Sequence

Register	Balanced Mode (hex)	Low Power Mode (hex)	Decimal
x1	0x000000C8	0x000000C8	200
x2	0x64646400	0x64646400	1684300800
x3	0x00000064	0x00000064	100
x4	0x000000C8	0x000000C8	200
x5	0x000000C8	0x000000C8	200
x6	0x000000C8	0x000000C8	200
x7	0x00000000	0x00000000	0
x28	0x000000C8	0x000000C8	200
x29	0x00000000	0x00000000	0
x30	0x000000C8	0x000000C8	200
Total Cycles	**49**	**41**	**16.3% reduction**

Table 14. Configuration Selection Guidelines

Workload	Recommended Mode	Rationale
Image-intensive	3-stage	16.3% faster
Arithmetic-heavy	Either mode	Identical cycle counts
Control-intensive	5-stage	Higher max frequency capability
Low activity standby	3-stage	Reduced static power

Note: At identical operating frequency (100 MHz), both pipeline modes achieve the same cycle counts for arithmetic and mixed workloads. The 5-stage configuration enables higher maximum frequency operation because its stages have shorter critical paths. The 3-stage mode shows its advantage for image-heavy tasks with a 16.3% cycle reduction.

The combined benefit of the two-level reconfiguration can be inferred from Table 12 and Table 13. The ISA extensions contribute a 10.28× EDP improvement while the pipeline reconfiguration achieves a 16.3% cycle reduction for image-intensive workloads. These two levels of optimization deliver both benefits simultaneously.

FPGA Synthesis Details

The design was implemented on the Xilinx Artix-7 XC7A200T board using Vivado post-synthesis (RTL verification).

Complete Resource Utilisation

Table 15 shows that the design utilises only 2.21% of LUTs and 0.69% of flip-flops, demonstrating minimal hardware requirements. The absence of DSP blocks shows that the custom instructions use logic-based implementation, allowing them to operate on low-cost FPGA systems. The details of the logic based implementation and the individual resource utilization for custom instructions is reported in the Design and Implementation section.

Table 15. Resource Utilization Summary

Resource	Used	Available	Utilisation
Slice LUTs	2,971	134,600	2.21%
Flip-Flops	1,858	269,200	0.69%
Block RAM	0	365	0.00%
DSP Blocks	0	740	0.00%
F7 Muxes	360	67,300	0.53%
F8 Muxes	160	33,650	0.48%
I/O Pins	3	400	0.75%

Power Analysis

Power Analysis shows a total on-chip power consumption of 0.124 W, with 98% from static power and 2% from dynamic switching activity. The junction temperature stays at 25.2°C and the thermal margin reaches 59.8°C, demonstrating thermally safe operation. Table 16 shows the breakdown of total on-chip power.

ARTICLE IN PRESS

Table 16. On-Chip Power Analysis Summary (Xilinx Artix-7, 100 MHz)

Category	Sub-category	Power (W)	Share (%)
Dynamic (2%)	Clocks	0.002	68
	Signals	<0.001	16
	Logic	<0.001	13
	I/O	<0.001	3
Static (98%)	Device	0.122	98
Total On-Chip	–	**0.124**	**100**
Junction temperature: 25.2°C		Thermal margin: 59.8°C	

Timing Analysis

Table 17 shows the post-implementation timing analysis indicating that all user-defined timing constraints are satisfied with a Worst Negative Slack (WNS) of 6.898 ns and zero failing endpoints for both setup and hold paths. The design reports no total negative slack, indicating successful timing closure and stable operation at the 100 MHz target clock frequency.

Table 17. Post-Implementation Timing Analysis (100 MHz target)

Check	Metric	Value (ns)	Failing Endpoints
Setup	Worst Negative Slack (WNS)	6.898	0
Setup	Total Negative Slack (TNS)	0.000	0
Hold	Worst Hold Slack (WHS)	0.188	0
Hold	Total Hold Slack (THS)	0.000	0
Pulse Width	Worst Pulse Width Slack (WPWS)	4.500	0
Total endpoints		143 (setup/hold), 112 (pulse width)	
All user-specified timing constraints met			

ARTICLE IN PRESS

Limitations and Future Work

The present system shows notable improvements but it also presents some operational limitations. The custom instruction set provides functionality for specific image-processing tasks through its extended ISA. The current system needs additional hardware components to achieve advanced kernel processing capabilities. The histogram Equalization technique needs a LookUp Table (LUT) to be stored because this Pre-fetching method avoids delays due to external memory access. The static power consumption of the Artix-7 platform establishes the primary power consumption pattern, making it difficult to determine the dynamic power efficiency advantages from custom instruction operations.

The future research will focus on expanding the instruction set architecture through additional image-processing instructions including convolution, Gaussian filtering, and edge-preserving filters. System performance will reach higher efficiency levels through DSP block implementations that can handle multiply and accumulate operations. The transition to an ASIC design process will provide better assessment of power consumption and space efficiency by removing FPGA static power consumption. The combination of on-chip memory buffers will enable better scalability for real-time high-resolution image processing.

Conclusion

The study demonstrates a multi-level reconfigurable RISC-V system which supports custom image-processing instructions and shows capacity for both fixed and adaptive pipeline depths. The design achieves a performance increase of 5.4× while reducing instructions by 16.7× through branch penalty elimination and substantial code size reduction.

The Kvasir gastrointestinal dataset demonstrated perfect clinical validation results, achieving 100% accuracy in eight distinct test cases. The entire system executes edge-based polyp detection at 3.45 μs per 8×8 region (345 cycles), allowing real-time capsule endoscopy operation processing 640×480 frames at 60.4 FPS. The system operates at 124 mW power consumption while achieving 0.428 nJ energy efficiency per classification.

The Xilinx Artix-7 XC7A200T FPGA implementation shows 2.21% LUT utilisation, achieves successful timing closure without any violations, and maintains stable performance at 0.124 W total on-chip power. The research results show that embedded image-processing workloads gain performance and reliability improvements through the combination of domain-specific ISA extensions and pipeline-level reconfiguration without any negative impact on hardware efficiency.

ARTICLE IN PRESS

Acknowledgements

Author Contributions Statement

Amritha Anand Deisgn Simulation & Manuscript preparation
Saraswathy B : Facilitate tool usage and verification of results
All authors reviewed the manuscript.

Data Availability

The Kvasir dataset (version 1) used in this study is publicly available at https://datasets.simula.no/kvasir/

Additional Information

Competing interests: The authors declare no competing interests.

Funding Information

No funding was obtained.

References

1. Rezunov, E., Zurstraßen, N., Reimann, L. M. & Leupers, R. Automatic microarchitecture-aware custom instruction design for RISC-V processors. *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.* **42**, 1–14 (2023).
2. Li, R., Peng, P., Shao, Z., Jin, H. & Zheng, R. Evaluating RISC-V vector instruction set architecture extension with computer vision workloads. *J. Comput. Sci. Technol.* **38**, 779–797 (2023).
3. Luo, Q. & Wu, K. A RISC-V based image acquisition system using CIS for industrial print quality inspection. *J. Eng. Res.* DOI: 10.1016/j.jer.2025.10.002 (2025).
4. Baglietto, P., Maresca, M., Migliardi, M. & Zingirian, N. Image processing on high performance RISC systems. *Proc. IEEE* **84**, 917–930 (1996).
5. Dörflinger, A. *et al.* A comparative survey of open-source application-class RISC-V processor implementations. In *Proc. 18th ACM Int. Conf. Comput. Frontiers (CF '21)* 12–20 (2021).
6. Gholizadehazari, M., Mahjoori, A. A. & Pedram, H. An FPGA implementation of a RISC-V based SoC system for image processing applications. In *Proc. 29th Iranian Conf. Elect. Eng. (ICEE)* 1–6 (2021).
7. Liu, J., Huang, Z., Cai, G., Wang, X. & Liu, Y. Custom instruction design for image preprocessing in RISC-V. In *Proc. 7th Int. Conf. Artif. Intell. Pattern Recognit. (AIPR '24)* 649–655 (2024).
8. Lima, A. *et al.* A Heterogeneous Multi-RISCV Architecture for Image Processing and Computer Vision Applications. Preprint at ResearchGate (2024).
9. Kumar M, A., Kumar, V., John, D. & Shanker, S. Implementation and analysis of custom instructions on RISC-V for edge-AI applications. In *Proc. 14th Int. Symp. Highly Efficient Accelerators Reconfigurable Technol. (HEART)* (2023).
10. Cui, E., Li, T. & Wei, Q. RISC-V Instruction Set Architecture Extensions: A Survey. *IEEE Access* **11**, 1–1 (2023).
11. Zhang, W., Zhang, Y. *et al.* Three-stage pipeline RISC-V processor. Tech. Rep. (2023).
12. Chen, T. *et al.* PipeCIM: A High-Throughput Computing-In-Memory Microprocessor With Nested Pipeline. *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.* (2024).
13. Satyanarayana, B., Madhu, N. & Srinivas, K. Design and Implementation of High-Performance RISC-V Processor with Five Stage Pipeline on FPGA. In *Proc. ICSSAS 2025* 68–75 (2025).
14. Ben-Asher, Y. & Qashqoush, I. An Extended pipeline RISC-V CPU architecture. In *Proc. MCSoC 2024* 517–523 (2024).
15. Saussereau, J., Jego, C., Leroux, C. & Begueret, J.-B. Design and Implementation of a RISC-V core with a Flexible Pipeline for Design Space Exploration. In *Proc. NewCAS 2023* 1–5 (2023).
16. Dao, N., Attwood, A., Healy, B. & Koch, D. FlexBex: A RISC-V with a Reconfigurable Instruction Extension. In *Proc. ICFPT 2020* (2020).
17. Mohan, P., Das, S. & Mai, K. A RISC-V SoC With Reconfigurable Custom Instructions on a Synthesized eFPGA Fabric in 22 nm FinFET. *IEEE Solid-State Circuits Lett.* **8**, 1–1 (2025).
18. Scheipel, T., Angermair, F. & Baunach, M. moreMCU: A Runtime-reconfigurable RISC-V Platform for Sustainable Embedded Systems. In *Proc. DSD 2022* (2022).

ARTICLE IN PRESS

ARTICLE IN PRESS

19. Charaf, N., Kamaleldin, A., Thuemmler, M. & Göhringer, D. RV-CAP: Enabling Dynamic Partial Reconfiguration for FPGA-Based RISC-V System-on-Chip. In *Proc. IPDPSW 2021* (2021).

20. Nolting, S., Hoyer, I., Utz, A., Kappert, H. & Grau, G. Developing Custom RISC-V ISA Extensions for General Embedded Image Processing Operations. In *RISC-V Summit Europe, Barcelona* (2023).

ARTICLE IN PRESS